

VSB ENGINEERING COLLEGE,KARUR

Name : S.Ponmithra
Class : cse B
Reg no : 922525104113

STUDENT MANAGEMENT SYSTEM

CRUD-Based Web Application Development

PROJECT REPORT

1. Title of the Project

Student Management System Using React, Django REST Framework and SQLite

2. Abstract

The Student Management System is a full-stack web application developed to manage student information efficiently. The application provides the basic CRUD operations: Create, Read, Update, and Delete. It consists of a React-based frontend, Django REST Framework backend, and SQLite database.

The system allows users to add new student records, view existing records, update student information, delete records, and search for students. Input validation is implemented on both the frontend and backend to maintain accurate and reliable data. REST APIs are used for communication between the frontend and backend.

This project demonstrates the practical implementation of frontend development, backend development, REST API integration, database management, validation, testing, and version control.

3. Introduction

Managing student information manually can be time-consuming and may result in errors, duplicate records, and difficulty in maintaining data. A computerized Student Management System provides an efficient solution for storing and managing student details.

The proposed system provides a simple web interface through which student records can be managed. The React frontend communicates with the Django REST API using Axios. Django processes the requests and uses the Django ORM to communicate with the SQLite database.

The system is designed to be simple, responsive, user-friendly, and suitable for educational institutions and academic projects.

4. Problem Statement

Traditional methods of maintaining student records may involve paper files or separate documents. These methods can make it difficult to:

- Add new student information
- Search existing records
- Modify incorrect information
- Remove outdated records
- Maintain data consistency
- Quickly retrieve student details

Therefore, a web-based Student Management System is developed to provide a centralized and efficient method for managing student information.

5. Objectives

The main objectives of the project are:

1. To develop a web-based Student Management System.
2. To implement Create, Read, Update, and Delete operations.
3. To store student information in a database.
4. To provide a simple and responsive user interface.
5. To implement REST APIs using Django REST Framework.
6. To validate user input on both frontend and backend.
7. To provide student search functionality.
8. To handle errors and invalid requests properly.
9. To test the application and API endpoints.
10. To understand frontend-backend-database integration.

6. Scope of the Project

The project can be used to manage basic student information.

The scope includes:

- Student registration
- Student record viewing

- Student record modification
- Student record deletion
- Student searching
- Form validation
- REST API communication
- Database storage
- Error handling
- API testing

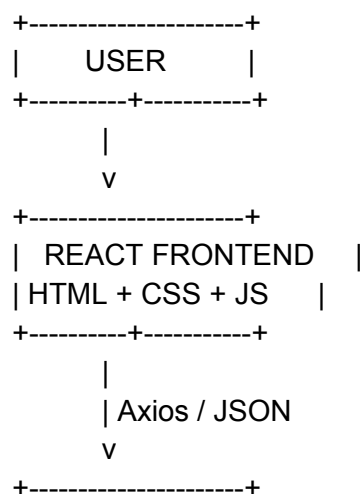
The application can later be extended with authentication, roles, attendance, marks, courses, and other academic features.

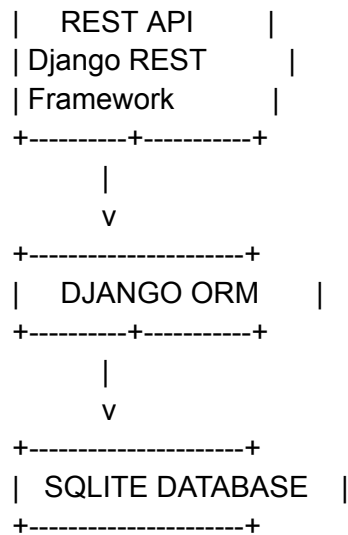
7. Technology Stack

Technology| Purpose
 HTML| Structure of the web interface
 CSS| Styling and responsive design
 JavaScript| Frontend programming
 React| Component-based frontend
 Axios| API communication
 Python| Backend programming
 Django| Backend framework
 Django REST Framework| REST API development
 SQLite| Database
 Postman| API testing
 Git/GitHub| Version control

8. System Architecture

The application follows a three-layer architecture.





The user interacts with the React frontend. The frontend sends requests to the Django REST API using Axios. Django processes the request and uses the ORM to communicate with SQLite. The result is returned to the frontend as JSON data.

9. Modules of the System

The application consists of the following modules:

9.1 Student Registration Module

This module allows users to enter and save student details such as name, email, phone, department, year, and address.

9.2 Student List Module

This module displays all the student records stored in the database.

9.3 Student Search Module

Users can search for students using their name, email, or department.

9.4 Student Update Module

Existing student information can be edited and updated.

9.5 Student Delete Module

Users can delete an unwanted student record after confirmation.

9.6 Validation Module

The system validates required fields, email format, phone number, and academic year.

9.7 REST API Module

The backend provides REST API endpoints for performing CRUD operations.

9.8 Database Module

SQLite stores the student records persistently.

10. Database Design

The system uses a Student table.

Student Table

Field	Data Type	Constraint
id	Integer	Primary Key
name	String	NOT NULL
email	Email/String	UNIQUE, NOT NULL
phone	String	NOT NULL
department	String	NOT NULL
year	Integer	NOT NULL
address	Text	Optional

ER Representation

```
+-----+
|      STUDENT      |
+-----+
| PK id              |
| name               |
| email (UNIQUE)     |
| phone              |
| department         |
| year               |
| address            |
+-----+
```

The "id" uniquely identifies each student. The email field is unique to prevent duplicate student records.

11. CRUD Operations

CRUD stands for:

- C – Create
- R – Read
- U – Update
- D – Delete

Create

A user enters valid student information and submits the form. The information is sent to the backend through a POST request and stored in the database.

Read

The application retrieves student records from the database using a GET request and displays them in the frontend.

Update

The user selects an existing student and edits the required information. The updated data is sent to the backend using PUT or PATCH.

Delete

The user selects a student and confirms deletion. A DELETE request is sent to the backend and the record is removed from the database.

12. REST API Endpoints

Operation	HTTP Method	Endpoint
Create Student	POST	"/api/students/"
Read All Students	GET	"/api/students/"
Read One Student	GET	"/api/students/<id>/"
Update Student	PUT	"/api/students/<id>/"
Partial Update	PATCH	"/api/students/<id>/"
Delete Student	DELETE	"/api/students/<id>/"

The APIs exchange data in JSON format.

13. Sample Create Request

A sample POST request contains:

```
{
```

```
"name": "Arun Kumar",  
"email": "arun@example.com",  
"phone": "9876543210",  
"department": "CSE",  
"year": 2,  
"address": "Coimbatore"  
}
```

If the information is valid, the server stores the record and returns a successful response.

14. Frontend Implementation

The frontend is developed using React.

The main components include:

- "App.jsx"
- "StudentForm.jsx"
- "StudentList.jsx"
- "StudentItem.jsx"
- "api.js"
- "index.css"

Student Form

The form collects:

- Student name
- Email
- Phone
- Department
- Year
- Address

The form performs client-side validation before sending information to the backend.

Student List

The student list displays records in a table and provides Edit and Delete buttons.

15. Backend Implementation

The backend is developed using Django and Django REST Framework.

The backend contains:

- Django Model
- Serializer
- Views/API
- URL configuration
- Database
- Validation
- Error handling

The Django model represents the Student database table.

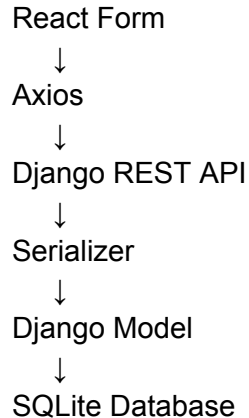
The serializer converts Django model objects into JSON and validates incoming data.

The API views process HTTP requests and perform CRUD operations.

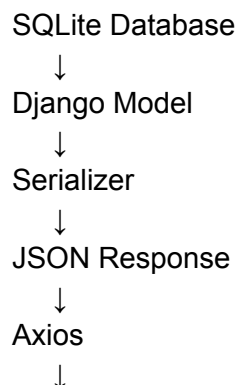
16. Frontend and Backend Integration

Axios is used to connect the React frontend with the Django backend.

The communication process is:



When the database operation is completed, the response is returned to React.



React

This allows the interface to update dynamically without manually modifying the database.

17. Validation

Validation is implemented on both frontend and backend.

The system checks that:

- Required fields are not empty.
- Email has a valid format.
- Email is not duplicated.
- Phone number contains valid digits.
- Department is provided.
- Year is within the valid range.
- Invalid records are rejected.

Server-side validation is important because frontend validation alone cannot guarantee data integrity.

18. Search Functionality

The system provides a search box to find student records.

The search can be performed using:

- Name
- Email
- Department

When the user enters a search value, matching records are displayed.

If no matching record is available, the system displays:

"No matching students found."

19. Error Handling

The application provides appropriate error handling for different situations.

Examples include:

- Invalid student data
- Duplicate email
- Student not found
- Network failure
- Backend unavailable
- Invalid student ID
- Database-related errors

Meaningful messages are displayed to the user instead of technical error details.

20. Testing

Testing is performed to verify that the application works correctly.

Test Cases

Test Case| Input/Action| Expected Result

Create Student| Valid student data| Student created

Create Student| Empty required field| Validation error

Create Student| Duplicate email| Error displayed

Read Students| Open student list| Records displayed

Read Student| Valid ID| Student details displayed

Read Student| Invalid ID| Not found error

Update Student| Valid updated data| Record updated

Update Student| Invalid data| Validation error

Delete Student| Valid ID| Record deleted

Delete Student| Invalid ID| Error displayed

Search| Existing student name| Matching record displayed

Search| Unknown name| No matching record message

21. API Testing Using Postman

Postman is used to test the REST API independently from the frontend.

The following operations are tested:

1. POST – Create student
2. GET – Retrieve all students
3. GET – Retrieve one student
4. PUT – Update student
5. PATCH – Partially update student
6. DELETE – Delete student

Invalid inputs are also tested to verify backend validation.

22. Security and Quality

The following security practices are followed:

- User input is validated.
- Django ORM is used for database operations.
- Passwords and secret keys are not hard-coded.
- Sensitive configuration should be stored using environment variables.
- Frontend and backend responsibilities are separated.
- Meaningful variable and function names are used.
- Unnecessary sensitive information is not exposed to users.

23. Version Control

Git is used to manage the source code.

Example commands:

```
git init
git add .
git commit -m "Initial project setup"
git commit -m "Add student CRUD API"
git commit -m "Add React frontend"
git commit -m "Integrate frontend and backend"
git commit -m "Add validation and testing"
```

The completed project can be uploaded to GitHub for source-code management and submission.

24. Advantages

The Student Management System provides several advantages:

- Easy student record management
- Faster data retrieval
- Simple user interface
- Reduced manual work
- Data stored systematically
- Easy modification of records
- Easy deletion of records

- Search functionality
- Validation of incorrect data
- REST API-based architecture

25. Limitations

The current version has some limitations:

- It uses SQLite for database storage.
- Authentication is not included.
- Role-based access is not implemented.
- The system manages basic student information only.
- Advanced reporting features are not included.

These features can be added in future versions.

26. Future Enhancements

The project can be enhanced by adding:

- User login and authentication
- Admin and student roles
- Attendance management
- Marks management
- Course management
- Pagination
- Advanced filtering
- CSV/PDF export
- Email notifications
- PostgreSQL or MySQL database
- Cloud deployment

27. Installation and Execution

Backend Setup

```
cd backend
python -m venv venv
```

Activate the virtual environment and install dependencies:

```
pip install -r requirements.txt
```

Run migrations:

```
python manage.py makemigrations  
python manage.py migrate
```

Start the backend:

```
python manage.py runserver
```

Frontend Setup

Open another terminal:

```
cd frontend  
npm install  
npm run dev
```

The React frontend communicates with the Django REST API running on the local server.

28. Expected Output

After starting the application, the user can:

1. Open the Student Management System.
2. Enter student information.
3. Click Add Student.
4. View the newly created record.
5. Search for a student.
6. Click Edit to modify information.
7. Save the updated record.
8. Click Delete to remove a record.

Thus, all four CRUD operations can be demonstrated successfully.

29. Learning Outcomes

After completing this project, the student will be able to:

- Understand full-stack web application architecture.
- Develop React components.
- Create responsive web interfaces.
- Develop Django REST APIs.
- Work with Django models and serializers.

- Perform database CRUD operations.
- Connect frontend and backend using Axios.
- Implement client-side and server-side validation.
- Test APIs using Postman.
- Handle application errors.
- Use Git for version control.
- Document and demonstrate a software project.

30. Conclusion

The Student Management System successfully demonstrates the development of a complete CRUD-based full-stack web application. React is used to create the frontend interface, while Django REST Framework provides the backend REST APIs. SQLite is used to store student information, and Axios connects the frontend with the backend.

The application supports creating, viewing, updating, deleting, and searching student records. Validation, error handling, API testing, and database integration are also included.

This project provides practical knowledge of modern web application development and demonstrates how frontend, backend, REST API, and database components work together to create a functional software system.

31. Final Demonstration Checklist

Requirement	Status
Application starts successfully	✓
Database connection works	✓
Create operation	✓
Read operation	✓
Update operation	✓
Delete operation	✓
Validation	✓
Search	✓
REST API	✓
Error handling	✓
Postman testing	✓
Git repository	✓
Documentation	✓

32. Viva Questions

1. What is CRUD?

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations performed on stored data.

2. Why is React used?

React is used to create a dynamic and component-based frontend user interface.

3. Why is Django REST Framework used?

Django REST Framework is used to create RESTful APIs that allow the frontend to communicate with the backend.

4. Why is Axios used?

Axios is used to send HTTP requests from the React frontend to the Django backend.

5. Why is SQLite used?

SQLite is a lightweight relational database that is simple to configure and suitable for small academic projects.

6. What is a REST API?

A REST API is an interface that allows applications to communicate using HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

7. What is a serializer?

A serializer converts complex Django objects into JSON and also validates and converts incoming JSON data.

8. What is an ORM?

ORM stands for Object-Relational Mapping. Django ORM allows developers to interact with the database using Python objects instead of writing raw SQL for common operations.

9. What is CORS?

CORS stands for Cross-Origin Resource Sharing. It allows a frontend running on one origin to communicate with a backend running on another permitted origin.

10. What is the purpose of Postman?

Postman is used to send API requests and verify API responses during testing.

33. Project Summary

Project Name: Student Management System

Frontend: React, HTML, CSS, JavaScript

Backend: Django REST Framework

Database: SQLite

API Communication: Axios

API Testing: Postman

Version Control: Git/GitHub

Main Function: Complete CRUD-based student record management